Experiment-6

Recurrent Neural Network

1. Aim

To study and implement a vanilla Recurrent Neural Network (RNN) for sequence modelling by training a character-level language model on a small excerpt of the Tiny Shakespeare dataset, and to understand sequence unrolling and hidden-state evolution in RNNs.

2. Theory

Recurrent Neural Networks (RNNs) are a class of deep learning models specifically designed to process and model sequential data. RNNs originated as models inspired by biological neural systems and were initially explored within cognitive science and neuroscience. Over time, they were adopted by the machine learning community as powerful tools for modelling sequential data.

Unlike traditional feedforward neural networks, which assume inputs to be independent of one another, RNNs incorporate recurrent connections that allow information to persist across time steps. These recurrent connections can be visualized as cycles in the network architecture, enabling the model to retain and utilize information from previous inputs while processing the current input.

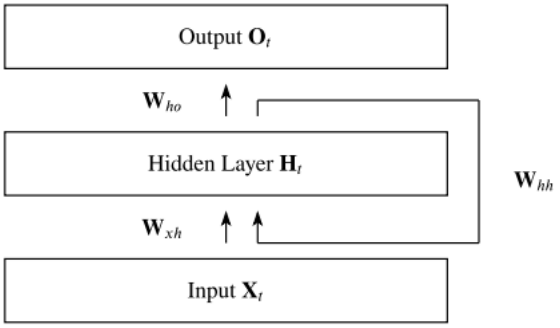

Fig. 1. Visualization of Recurrent Neural Network

(Source: Robin M. Schmidt, *"Recurrent Neural Networks (RNNs): A Gentle Introduction and Overview,"* arXiv:1912.05911v1, 2019)

<u>Mathematical Representation of Recurrent Neural Network</u>

An RNN processes an input sequence one element at a time while maintaining a hidden state that captures information from previous time steps. At each time step *t*, the hidden state H_t is updated using the current input X_t and the hidden state from the previous time step H_{t-1}.

The hidden state update is given by:

$$H_t = \varphi_h(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

where:

- X_t is the input at time step *t*
- H_t is the hidden state at time step *t*

- W_{xh} is the input-to-hidden weight matrix
- W_{hh} is the hidden-to-hidden (recurrent) weight matrix
- b_h is the bias vector
- $\varphi_h(\cdot)$ is the activation function

The output O_t at time step *t* is computed as:

$$O_t = \varphi_o(H_t W_{ho} + b_o)$$

where:

- W_{ho} is the hidden-to-output weight matrix
- b_o is the output bias

This process is repeated across all time steps, with the same parameters shared throughout the sequence, enabling the RNN to model temporal dependencies effectively.

Time Unrolling and Parameter Sharing

At an initial observation, the presence of cycles may appear to contradict the feedforward nature of neural networks, where computation flows strictly in one direction. However, RNNs resolve this apparent ambiguity through a precise formulation: the network is unrolled across time steps. In the unrolled representation, the RNN is transformed into a sequence of identical feedforward networks, one for each time step, where the same set of parameters is shared across all steps**.** This parameter sharing allows the model to generalize across sequences of varying lengths while maintaining temporal consistency.

Feedforward and Recurrent Connections

In an RNN, two types of connections operate simultaneously:

- **Standard (feedforward) connections** propagate activations from one layer to the next within the same time step.
- **Recurrent connections** transmit information from the hidden state at one time-step to the hidden state at the next.

Through this mechanism, the hidden state acts as a form of memory, capturing contextual information from earlier elements in the sequence and influencing future predictions.

Backpropagation Through Time

The gradient computation in Recurrent Neural Networks involves performing a forward propagation pass through the unrolled network, followed by a backward propagation pass. Due to the inherently sequential nature of recurrent computation, the runtime complexity is $O(\tau)$, where τ denotes the length of the input sequence. This computation cannot be parallelized across time steps, as each time step depends on the hidden state from the previous one.

During the forward pass, the hidden states at each time step must be stored so that they can be reused during the backward pass. As a result, the memory requirement is also $O(\tau)$. The application of the backpropagation algorithm to the unrolled RNN computation graph is known as Backpropagation Through Time (BPTT).

Merits of Recurrent Neural Networks:

- **Sequential Memory:**

RNNs retain information from previous inputs making them ideal for time-series predictions where past data is crucial. This makes them useful for the tasks such as language modelling, where the meaning of the word depends on the context in which it appears.

- **Ability to handle variable-length sequences:**
  RNNs are designed to handle the input sequences of variable length, which makes them well-suited for tasks such as speech recognition, natural language processing, and time-series analysis.

- **Parameter Sharing:**
  RNNs share the same set of parameters across all time steps, which reduce the number of parameters that need to be learnt and can lead to better generalization.

Demerits of Recurrent Neural Networks:

- **Vanishing Gradient:**
  During backpropagation, gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn long –term dependencies which is crucial for tasks like language translation.

- **Exploding Gradient:**
  Sometimes gradients grow uncontrollably causing excessively large weight updates that de-stabilize training, which leads to the problem of exploding gradient in RNNs.

- **Lack of Parallelism:**
  RNNs are inherently sequential, which makes it difficult to parallelize the computation. This can limit the speed and scalability of the network.

3. Pre-Test (MCQs)

1. What is the primary purpose of a Recurrent Neural Network (RNN)?
a) Image classification
(Incorrect because image classification is mainly handled by CNNs.)
b) Text generation
(Correct because RNNs are widely used for sequence tasks such as text generation.)
c) Reinforcement learning
(Incorrect because reinforcement learning is a learning paradigm, not a primary RNN purpose.)
d) Object detection
(Incorrect because object detection is a computer vision task.)
Answer: b

2. What is the main characteristic of a Recurrent Neural Network (RNN)?
a) It processes data sequentially and has loops in its architecture
(Correct because feedback loops enable sequence modelling.)
b) It processes data in parallel and has no loops
(Incorrect because that describes feedforward networks.)
c) It uses a single-layer architecture
(Incorrect because RNNs can have multiple layers.)
d) It is designed for image classification
(Incorrect because CNNs are used for images.)

Answer: a

3. Which layer type is typically used to capture sequential dependencies in an RNN?
a) Input layer
(Incorrect because the input layer only receives data.)
b) Hidden layer
(Correct because the hidden layer maintains memory across time steps.)
c) Output layer
(Incorrect because the output layer produces predictions, not dependencies.)
d) Activation layer
(Incorrect because activation layers add non-linearity but do not store sequence information.)
Answer: b

4. What is the advantage of using recurrent layers in an RNN?
a) They can capture temporal dependencies in the input data
(Correct because recurrent connections allow learning from past inputs.)
b) They can handle variable-length inputs
(Incorrect because this is a property of sequence models but not the main advantage.)
c) They can generate synthetic data
(Incorrect because data generation is an application, not an inherent advantage.)
d) They can handle non-linear transformations
(Incorrect because non-linearity comes from activation functions, not recurrence.)
Answer: a

5. What is the purpose of the hidden state in an RNN?
a) To store the information from the previous time step
(Correct because hidden state carries memory forward.)
b) To adjust the learning rate during training
(Incorrect because learning rate is controlled by the optimizer.)
c) To compute the gradients for backpropagation
(Incorrect because gradients are computed during backpropagation.)
d) None of the above
Answer: a

6. Which activation function is commonly used in the recurrent layers of an RNN?
a) ReLU
(Incorrect because ReLU is less stable for recurrent connections.)
b) Sigmoid
(Incorrec because sigmoid is mainly used in gates, not vanilla RNNs.)
c) Tanh (Hyperbolic Tangent)
(Correct because tanh keeps values bounded and centred.)
d) Softmax
(Incorrect because softmax is used in output layers.)
Answer: c

7. What is the purpose of the time step parameter in an RNN?
a) To determine the number of recurrent layers
(Incorrect because layers and time steps are different concepts.)
b) To adjust the learning rate
(Incorrect because learning rate is independent of time steps.)
c) To specify the length of the input sequence
(Correct because time steps define how many sequence elements are processed.)

d) None of the above
Answer: c

8. Which layer type is commonly used to initialize the hidden state in an RNN?
a) Input layer
(Incorrect because input layer does not initialize memory.)
b) Hidden layer
(Correct because the hidden state belongs to the hidden layer and is initialized at the start of sequence processing.)
c) Output layer
(Incorrect because output layer does not manage memory.)
d) Activation layer
(Incorrect because activation layers only apply non-linearity.)
Answer: b

9. What is the main difference between an RNN and a feedforward neural network?
a) RNN uses convolution layers
(Incorrect because convolution layers are used in CNNs, not RNNs.)
b) RNN has feedback connections
(Correct because RNNs contain recurrent (feedback) connections that allow information to persist across time steps.)
c) RNN has no hidden layers
(Incorrect because RNNs can have one or more hidden layers.)
d) RNN works only on images
(Incorrect because RNNs are mainly used for sequential data such as text and time series.)
Answer: b

10. What dataset is commonly used for character-level language modelling demonstrations?
a) MNIST
(Incorrect because MNIST is a handwritten digit image dataset, not used for text modelling.)
b) CIFAR-10
(Incorrect because CIFAR-10 is an image classification dataset.)
c) ImageNet
(Incorrect because ImageNet is used for large-scale image recognition tasks.)
d) Tiny Shakespeare
(Correct because Tiny Shakespeare is widely used for character-level language modelling with RNNs.)
Answer: d

4. Procedure

The objective of this part of the experiment is to implement a vanilla Recurrent Neural Network (RNN) for character-level sequence modelling. The Tiny Shakespeare dataset is used, where individual characters from a text corpus are learned to model sequential dependencies and generate coherent text. This part focuses on understanding sequence unrolling, hidden-state propagation across time steps, and the training of RNNs using Backpropagation Through Time (BPTT), along with visualizing the evolution of hidden states during text generation.

1. Import Required Libraries

   Import necessary Python libraries such as *PyTorch*, *Numpy*, and *Matplotlib* for model implementation, numerical operations, and visualizations.

2. Dataset Description and Loading

   The Tiny Shakespeare dataset is a character-level text corpus containing a small subset of William Shakespeare's plays. It consists of dialogues, character names, and stage directions written in plain text format.

   - Each character (letters, digits, punctuation, spaces) is treated as an individual token.
   - The dataset is suitable for sequence modelling and text generation tasks.
   - The text file is downloaded and loaded into memory for further pre-processing.

   The successful loading of the dataset is verified.

3. Dataset Splitting

   The text dataset is divided into three parts:

   - Training set (90%) – used for learning model parameters
   - Validation set (5%) – used to monitor overfitting
   - Test set (5%) – used for final evaluation

   This ensures proper training and evaluation of the model.

4. Vocabulary Creation and Encoding
   - Extract all unique characters to form the vocabulary.
   - Map each character to a unique integer index (character-to-index mapping).
   - Convert the entire text into a sequence of integers.
5. Hyper-parameter Initialization
   Initialize training parameters such as:

   - Number of epochs
   - Learning rate
   - Embedding size
   - Batch size
   - Sequence length
   - Hidden layer size
   - Number of RNN layers

   These parameters control learning behaviour and model capacity.

6. Batch Generation
   Create mini-batches of input sequences and corresponding target sequences using fixed sequence lengths (sliding window approach) to enable efficient training. Input sequences (x) and target sequences (y) are created such that each target character is the next character in the sequence.

7. RNN Model Definition

   Define a Character-level RNN model consisting of:

   - An Embedding layer to convert character indices into dense vectors
   - A multi-layer vanilla RNN to process sequential data
   - A Fully Connected (Linear) layer to predict the next character

   Initialize the model with zero hidden states.

8. Model Training:
   - Train the RNN using Backpropagation Through Time (BPTT).
   - Use Cross-Entropy Loss as the objective function.
   - Optimize the model using the Adam optimizer.
   - Apply gradient clipping to prevent exploding gradients.
   - Record training and validation loss after each epoch.
9. Loss Curve Visualization:
   Plot training and validation loss curves to analyse model convergence and learning behaviour.

10. Text Generation
    After training, generate new text by:
    - Providing a seed string
    - Predicting one character at a time
    - Feeding the previously generated character back into the model
11. Hidden State Visualization:
    - Extract hidden states for a short character sequence.
    - Plot the evolution of selected hidden units across time steps.
    - Analyse how the RNN maintains memory over the sequence.
12. Result Analysis:
    The generated text, loss curves, and hidden-state plots are analysed to evaluate the effectiveness of the RNN in learning sequential character-level patterns.

5. Post-Test (MCQs)

1. What is the purpose of the recurrent connection in an RNN?
a) To propagate the hidden state across different time steps
(Correct because this enables memory across the sequence.)
b) To adjust the weights and biases
(Incorrect because weight updates are handled during training.)
c) To reduce the dimensionality of the input
(Incorrect because dimensionality reduction is not the goal.)

d) None of the above
Answer: a

2. What is the purpose of the initial hidden state in an RNN?
a) To provide the starting point for the recurrent computation
(Correct because it initializes memory before sequence processing.)
b) To adjust the learning rate
(Incorrect because learning rate is not related to hidden state.)
c) To compute gradients
(Incorrect because gradients are computed during backpropagation.)
d) None of the above
Answer: a

3. How does the RNN handle sequential data?
a) Using a sliding window
(Incorrect because sliding windows do not retain memory.)
b) Processing data point by point with memory of past inputs
(Correct because hidden state retains past information.)
c) Averaging inputs
(Incorrect because this loses temporal order.)
d) Applying convolution
(Incorrect because convolutions are used in CNNs.)
Answer: b

4. In the context of RNNs, what does "unrolling" refer to?
a) Training with large batch size
(Incorrect because batch size is unrelated.)
b) Unfolding the RNN across time steps
(Correct because it helps visualize sequence processing.)
c) Reducing number of layers
(Incorrect because unrolling does not change depth.)
d) Implementing dropout
(Incorrect because dropout is a regularization technique.)
Answer: b

5. What is the primary reason for visualizing sequence unrolling in RNNs?
a) To reduce model complexity
(Incorrect because visualization does not change or reduce complexity.)
b) To understand how parameters change
(Incorrect because parameters are shared across time steps.)
c) To visualize how information flows over time
(Correct because unrolling shows how data and hidden states propagate across time steps.)
d) To improve accuracy directly
(Incorrect because visualization itself does not improve model accuracy.)
Answer: c

6. What happens if the initial hidden state of an RNN is set to zero?
a) The model cannot learn
(Incorrect because the model can still learn during training.)
b) The network starts without prior memory
(Correct because zero initialization means no past information at the start.)
c) The first output is ignored

(Incorrect because the first output is still computed.)
d) The gradients become zero permanently
(Incorrect because gradients are not permanently zero.)
Answer: b

7. Which visualization helps understand memory behaviour in RNNs?
a) Accuracy curve
(Incorrect because accuracy shows performance, not memory.)
b) Confusion matrix
(Incorrect because used for classification evaluation.)
c) ROC curve
(Incorrect because ROC curves are used for binary classification.)
d) Hidden-state evolution
(Correct because it shows how memory changes over time.)
Answer: d

8. During text generation, what is used as input after the first character?
a) Random noise
(Incorrect because noise is used in GANs, not RNN text generation.)
b) Original training data
(Incorrect because training data is not reused during generation.)
c) Previously generated character
(Correct because each generated character becomes the next input.)
d) Entire sentence
(Incorrect because generation is done step-by-step.)
Answer: c

9. What technique is used to train RNNs over multiple time steps?
a) Forward propagation
(Incorrect because forward propagation only computes outputs.)
b) Gradient descent
(Incorrect because gradient descent is an optimizer, not a sequence-specific method.)
c) Dropout
(Incorrect because dropout is a regularization technique.)
d) Backpropagation Through Time (BPTT)
(Correct because BPTT propagates gradients across time steps.)
Answer: d

10. What problem commonly occurs in vanilla RNNs when handling long sequences?
a) Overfitting
(Incorrect because overfitting is not specific to long sequences.)
b) Exploding memory
(Incorrect because memory does not explode; gradients may.)
c) Vanishing gradients
(Correct because gradients shrink over long sequences, affecting learning.)
d) Data leakage
(Incorrect because data leakage is a data handling issue.)
Answer: c

6. References

1. D. Rumelhart, G. Hinton & R. Williams, “Learning representations by back-propagating errors”, *Nature* 323, 533–536 (1986).

2. Robin M. Schmidt, “Recurrent Neural Networks (RNNs): A Gentle Introduction and Overview,” arXiv:1912.05911v1 (2019).

3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.